# SIMATS ENGINEERING
## Department of Computer Science & Engineering
### ASSESSMENT TOOL 1- EXPERIMENTS

| | | | |
|---|---|---|---|
| **Course Code:** | **CSA12** | **Course Title:** | **Computer Architecture** |
| **CO Assessed:** | **CO3** | **Assessment:** | **ASSESSMENT TOOL 1- EXPERIMENTS** |
| **Weightage:** | **40%** | **Total Marks:** | **25** |
| **CO3 Statement:** | *Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)* | | |
| **Student Name:** | **SHAIK HAZIPEERA** | **Reg.No.:** | **192572056** |
| **Department:** | **CSE(AI)** | **Date:** | **30-07-2026** |

## ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

***Code of Conduct:***

*I SHAIK HAZIPERA (192572056) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

*S. Hazipeera*
***Signature of the student:***

## MARKS ALLOCATION

| | Understanding of Concepts (20%) | Experimental Design (20%) | Use of Tools (25%) | Analysis of Results (20%) | Documentation (15%) |
|---|---|---|---|---|---|
| **Q1** | | | | | |
| **Q2** | | | | | |
| **Q3** | | | | | |
| **Q4** | | | | | |
| **Q5** | | | | | |

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 25 | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design | 30 | Well-planned, systematic implementation with optimal solution | Correct implementation with minor issues | Basic implementation with errors or inefficiencies | Incorrect or incomplete implementation |
| Use of Tools | 20 | Effective and advanced use of tools/technologies with proper configuration | Appropriate use of tools with correct execution | Limited or inconsistent use of tools | Incorrect or minimal use of tools |
| Analysis of Results | 15 | Insightful analysis with accurate interpretation and validation of results | Good analysis with reasonable interpretation | Basic analysis with limited insights | Poor or incorrect analysis of results |
| Documentation | 10 | Clear, well-structured documentation with diagrams, code, and results | Organized documentation with necessary details | Basic documentation with missing details | Poor or incomplete documentation |

# ANSWERS

## 1. Register Transfer Operations with ALU

### Aim

Design and simulate register transfers and ALU operations (ADD, SUB, AND, OR).

### Introduction

Registers are fast storage locations inside the CPU. Data is transferred between registers under the control of the control unit. The Arithmetic Logic Unit (ALU) performs arithmetic and logical operations on the transferred data.

### Theory

- Register transfer moves data between CPU registers.
- ALU performs addition, subtraction, AND and OR.
- Control signals determine the operation.
- Results are stored in the destination register.

### Illustrative Block Diagram

```
Register A ----\
               >---- Processing Unit / ALU / Pipeline ----> Output Register
Register B ----/
```

### Procedure

1. Load values into Register A and Register B.
2. Transfer operands to the ALU.
3. Select ADD/SUB/AND/OR.
4. Store result in Register C.
5. Verify outputs.

### Analysis

Using A=15 and B=10: ADD=25, SUB=5, AND=10, OR=15. The sequence demonstrates register transfer and ALU processing.

### Advantages

- Improves execution speed
- Efficient hardware utilization
- Reduces execution time
- Enhances processor performance

### Applications

- Microprocessors
- Embedded systems
- Computer architecture laboratories
- High-performance computing

### Conclusion

The simulation verifies correct register transfers and ALU functionality.

## 2. Single-Bus and Multi-Bus Organization

### Aim

Compare single-bus and multi-bus computer organizations.

### Introduction

A bus is a communication pathway connecting CPU, memory and I/O devices. Bus organization significantly affects system performance.

### Theory

- Single-bus architecture uses one shared bus.
- Multi-bus architecture allows simultaneous transfers.
- Multi-bus reduces bottlenecks and improves throughput.

### Illustrative Block Diagram

Register A ----\
>---- Processing Unit / ALU / Pipeline ----> Output Register
Register B ----/

### Procedure

6. Create both architectures.
7. Transfer data between registers, memory and I/O.
8. Compare transfer cycles and efficiency.

### Analysis

Single-bus systems are simpler but slower due to bus contention. Multi-bus systems support parallel transfers and achieve higher throughput.

### Advantages

- Improves execution speed
- Efficient hardware utilization
- Reduces execution time
- Enhances processor performance

### Applications

- Microprocessors
- Embedded systems
- Computer architecture laboratories
- High-performance computing

### Conclusion

Multi-bus organization provides better overall performance.

## 3. Five-Stage Instruction Pipeline

### Aim

Simulate a five-stage instruction pipeline.

## Introduction

Pipelining overlaps instruction execution to improve throughput.

## Theory

- Stages: IF, ID, EX, MEM, WB.
- Pipeline increases throughput after initial filling.
- Speedup depends on hazard frequency.

## Illustrative Block Diagram

Register A ----\
>---- Processing Unit / ALU / Pipeline ----> Output Register
Register B ----/

## Procedure

9. Execute instructions without pipelining.
10. Execute with pipelining.
11. Compare execution cycles.
12. Calculate throughput and speedup.

## Analysis

A pipelined processor completes more instructions per unit time than a non-pipelined processor.

## Advantages

- Improves execution speed
- Efficient hardware utilization
- Reduces execution time
- Enhances processor performance

## Applications

- Microprocessors
- Embedded systems
- Computer architecture laboratories
- High-performance computing

### Conclusion

Pipelining significantly improves processor efficiency.

## 4. Pipeline Hazards

### Aim

Study data, control and structural hazards and their solutions.

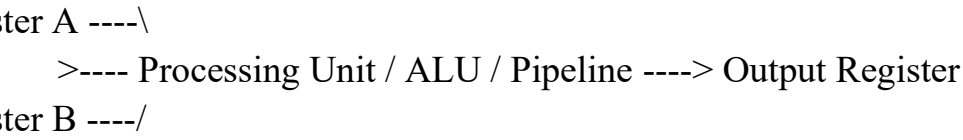
### Introduction

Hazards interrupt smooth pipeline execution and reduce performance.

### Theory

- Data hazards arise from dependencies.
- Control hazards arise from branches.
- Structural hazards occur due to resource conflicts.
- Forwarding, stalling and branch prediction reduce penalties.

### Illustrative Block Diagram




### Procedure

13. Create hazard scenarios.
14. Observe stalls.
15. Apply forwarding.
16. Apply branch prediction.
17. Compare results.

### Analysis

Forwarding minimizes data hazards, while branch prediction reduces control hazard penalties.

### Advantages

- Improves execution speed
- Efficient hardware utilization
- Reduces execution time
- Enhances processor performance

### Applications

- Microprocessors
- Embedded systems
- Computer architecture laboratories
- High-performance computing

### Conclusion

Hazard resolution techniques improve pipeline performance.

## 5. Superscalar, Out-of-Order Execution, Hyper-Threading and SMT

### Aim:

Analyze advanced processor execution techniques.

### Introduction:

Modern processors execute multiple instructions in parallel to maximize resource utilization.

### Theory

- Superscalar processors issue multiple instructions each cycle.
- Out-of-order execution avoids idle functional units.
- Speculative execution predicts future instruction paths.
- Hyper-threading and SMT execute multiple threads concurrently.

**Illustrative Block Diagram**

Register A ----\
>---- Processing Unit / ALU / Pipeline ----> Output Register
Register B ----/

**Procedure**

18. Study instruction issue.
19. Observe parallel execution.
20. Analyze resource utilization.
21. Compare throughput.

**Analysis**

These techniques increase instruction throughput and improve CPU utilization.

**Advantages**

- Improves execution speed
- Efficient hardware utilization
- Reduces execution time
- Enhances processor performance

**Applications**

- Microprocessors
- Embedded systems
- Computer architecture laboratories
- High-performance computing

**Conclusion**

Advanced execution mechanisms provide substantial performance gains over traditional processors.